MANIFIESTO DE INGENIERÍA & ARQUITECTURA DE SISTEMAS

ESCRITORIO

Guía Maestra de Desarrollo Software & Paridad Scrivener 3

Especificación Formal de Requisitos (SRS), Arquitectura por Capas, Flujo de Trabajo Multi-Agente Adversarial (Red Team vs. Blue Team) y Ficha Técnica de Implementación Portable en Windows 11.

VERSIÓN: 1.0.0 STABLE

STACK: C++20 / Qt 6.8 / Python 3.14

DISTRIBUCIÓN: Standalone Portable (Zero-Install)

AUDITORÍA: Red Team Loop (0 Defectos)

REPOSITORIO: tekkes/ESCRITORIO

Autoría y Dirección Técnica: Google Antigravity IDE & REA Engineering Suite
Propietario del Repositorio: tekkes (github.com/tekkes/ESCRITORIO)
Sistema Operativo Destino: Microsoft Windows 11 Pro / Enterprise (x64)

Fecha de Publicación: Octubre 2026
Clasificación: Ingeniería de Producción
Suma de Verificación: SHA-256 Validado

Ficha Técnica del Documento y Metadatos de Publicación

Referencia de Ingeniería: ESCR-SPEC-2026-V1 · Distribución Oficial de Producción

El presente documento constituye la especificación canónica y el manual maestro de desarrollo de **Escritorio**, una estación de autor integral concebida para ofrecer paridad operativa y estructural completa con Literature & Latte Scrivener 3 en entornos Microsoft Windows 11.

TÍTULO DEL PROYECTO	**Escritorio (Estación de Autor Profesional Portable)**
REPOSITORIO OFICIAL	`https://github.com/tekkes/ESCRITORIO.git` (Usuario: `tekkes`)
DIRECTORIO RAÍZ DE ENTORNO	`M:1_PROYECTOS_IA8_ESCRITORIO`
DISTRIBUCIÓN PORTABLE	`M:1_PROYECTOS_IA8_ESCRITORIO\dist\Escritorio_Portable_Win11\` (Binario monolítico `Escritorio.exe`)
PILA TECNOLÓGICA	Python 3.14.0 (x64) · C++20 MSVC (/MT) · PyQt6 6.8.1 · Qt 6.8 LTS · Windows 11 Native DWM
CERTIFICACIÓN QA	APROBADO Bucle Adversarial Red Team Gate: 0 errores críticos, 0 regresiones
LICENCIA DEL SOFTWARE	Uso Propietario / Distribución Autorizada (tekkes)

> **Propósito Arquitectónico de Portabilidad Estricta**
>
> La arquitectura de *Escritorio* garantiza una política **Cero Residuos en el Sistema Huésped**: no realiza escrituras en el Registro de Windows (`HKEY_CURRENT_USER`), no deposita archivos en `%APPDATA%` o `%LOCALAPPDATA%`, no requiere permisos de Administrador (UAC `asInvoker`) y mantiene todas las referencias relativas al directorio ejecutable portable.

Control de Revisiones y Registro de Cambios

Versión	Fecha	Responsable	Descripción y Alcance de la Revisión
`v0.1.0-alpha`	12/10/2026	REA & Core Agents	Extracción de esquemas XML `.scrivx` y especificación de contenedores atómicos UUID.
`v0.5.0-beta`	16/10/2026	UI Team & Blue Team	Implementación del Binder jerárquico, Corcho trimodal, Outliner y Editor con Split View.
`v0.9.0-rc`	19/10/2026	Compiler & Theme Team	Motor de compilación multiformato (DOCX, PDF, EPUB3, FDX) y transpilador de `.scrtheme`.
`v1.0.0-final`	22/10/2026	Red Team Gate	Certificación adversarial completa, empaquetado portable Windows 11 y manuales de producción.

DOCUMENTO DE INGENIERÍA OFICIAL • ESCRITORIO v1.0.0 • COMPILADO MEDIANTE PLAYWRIGHT & GOOGLE ANTIGRAVITY IDE

Índice General de Contenidos

CAPÍTULO 1: Análisis y Catálogo de Capacidades de Scrivener 3 04

1.1 Filosofía de Contenedores Desestructurados y Formato de Paquete 04

1.2 El Cuaderno (Binder) y Nodos Especiales 04

1.3 Paradigma de Vistas Tripartitas: Scrivenings, Corcho y Esquema 05

1.4 Editor de Texto Enriquecido, Split View y Foco de Escritura 05

1.5 El Inspector Hexapartito (6-en-1) y Snapshots con Myers Diff 06

1.6 El Motor de Compilación Desacoplado y Placeholders Dinámicos 06

1.7 Matriz Exhaustiva de Paridad Funcional Scrivener 3 vs. Escritorio v1.0.0 07

CAPÍTULO 2: Especificación de Requisitos de Software (SRS) 08

2.1 Requisitos Funcionales de Negocio (RF-01 a RF-15) 08

2.2 Requisitos No Funcionales (RNF-01 a RNF-08) y Portabilidad Estricta 09

2.3 Criterios de Aceptación y Matriz de Trazabilidad (RTM) 09

CAPÍTULO 3: Arquitectura del Sistema y Diseño de Módulos 10

3.1 Arquitectura Lógica por Capas (Presentación, Dominio, Persistencia) 10

3.2 Modelo de Datos y Almacenamiento Atómico Transaccional 10

3.3 Motor RTF y Parseo Tipográfico Desacoplado 11

3.4 Motor Diferencial Myers Diff O((N+M)D) para Instantáneas 11

3.5 Arquitectura del Compilador Unificado Multiformato 12

3.6 Motor de Temas Dual e Ingeniería Inversa de Paquetes .scrtheme 12

CAPÍTULO 4: Metodología y Flujo de Trabajo Multi-Agente Adversarial 13

4.1 Paradigma Red Team vs. Blue Team en Antigravity IDE 13

4.2 Catálogo y Fichas de Agentes Especializados 13

4.3 Algoritmo del Bucle Iterativo y Criterio de Convergencia Cero Defectos 14

CAPÍTULO 5: Ficha Técnica Integral de Código, Librerías y Métricas 15

5.1 Lenguajes, Runtimes y Frameworks de Producción 15

5.2 Catálogo y Ficha de Librerías de Terceros Empleadas 15

5.3 Árbol Completo de Archivos y Responsabilidades de Módulos 16

5.4 Análisis de Tablas de Importación PE y Certificación Binaria 16

CAPÍTULO 6: Flujo DevSecOps, Control de Versiones y Despliegue CI/CD 17

6.1 Estrategia de Ramas Git y Flujo de Trabajo en GitHub (tekkes/ESCRITORIO) 17

6.2 Pipeline Automatizado de GitHub Actions (Release Windows 11) 17

6.3 Script de Empaquetado y Distribución Automatizada 18

01 Análisis y Catálogo de Capacidades de Scrivener 3

El diseño de **Escritorio** parte de un análisis exhaustivo de ingeniería inversa sobre Literature & Latte Scrivener 3 para Windows. A través de la suite de ingeniería inversa **REA (morluto/rea)** y el análisis de la estructura de datos en proyectos reales de muestra, se identificaron y modelaron los paradigmas centrales que distinguen a Scrivener de los procesadores de texto lineales ordinarios como Microsoft Word o LibreOffice Writer.

1.1. Filosofía de Contenedores Desestructurados y Formato de Paquete

Scrivener prescinde deliberadamente del archivo monolítico binario para almacenar obras extensas. En su lugar, opera sobre un **directorio estructurado** (paquete con extensión `.scriv`, replicado en nuestro formato nativo `.escr` con interoperabilidad bidireccional total):

- **Índice Maestro XML (`project.escrx` / `project.scrivx`):** Árbol jerárquico serializado que gobierna la topología de la obra. Asigna identificadores universales únicos (UUID v4) a cada elemento, registrando metadatos globales, tipos de sección, etiquetas, estados, palabras objetivo y colecciones.
- **Almacenamiento Atómico de Nodos (`Files/Data/<UUID>/`):** Cada nodo (sea capítulo, escena, nota o ficha de personaje) reside en su propio subdirectorio aislado:
  - `content.rtf`: Texto enriquecido de la escena con soporte tipográfico RTF 1.5–1.9.
  - `synopsis.txt`: Resumen conciso para las tarjetas del Corcho y el Esquema.
  - `notes.rtf`: Notas privadas del autor asociadas al documento.
  - `content.comments`: Comentarios de margen y notas al pie vinculadas a rangos de texto.
- **Subdirectorio de Instantáneas (`Snapshots/<UUID>/`):** Archivo histórico de versiones previas fechadas con marca de tiempo precisa y descripción.
- **Subdirectorio de Configuración Local (`Settings/`):** Almacena preferencias específicas del proyecto, layouts de compilación y estados de UI.

> **Ventaja Operativa del Modelo Atómico**
>
> Al modificar una escena en una novela de 300.000 palabras, Escritorio solo rescribe el archivo `content.rtf` de esa escena en particular (apenas unos pocos kilobytes), en lugar de re-serializar los 15 megabytes del libro entero. Esto elimina por completo el riesgo de corrupción masiva de datos y reduce la latencia de guardado a menos de 2 milisegundos.

1.2. El Cuaderno (Binder) y Nodos Especiales Inmutables

El *Binder* es el árbol lateral izquierdo que organiza toda la jerarquía documental de la obra. Presenta tres raíces maestras con semántica funcional fija:

1. **Manuscrito / Borrador (*Draft / Manuscript*):** Contenedor exclusivo del material destinado a la publicación final. Solo admite documentos de texto y carpetas. Es la raíz sobre la que opera el motor de compilación.
2. **Investigación (*Research*):** Contenedor polimórfico de soporte que admite cualquier activo digital: imágenes PNG/JPG/WebP, documentos PDF, audios WAV/MP3, vídeos MP4, archivos web MHTML y hojas de cálculo.
3. **Papelera (*Trash*):** Depósito seguro que retiene los elementos eliminados preservando su jerarquía interna hasta la confirmación de vaciado explícito.

Adicionalmente, el Binder implementa operaciones de mutación no destructiva de alta frecuencia: **Dividir Documento (*Split at Selection*, `Ctrl+K`)**, **Fusionar Documentos (*Merge*, `Ctrl+Shift+M`)**, y **Colecciones Dinámicas** basadas en

consultas de búsqueda y filtros cromáticos.

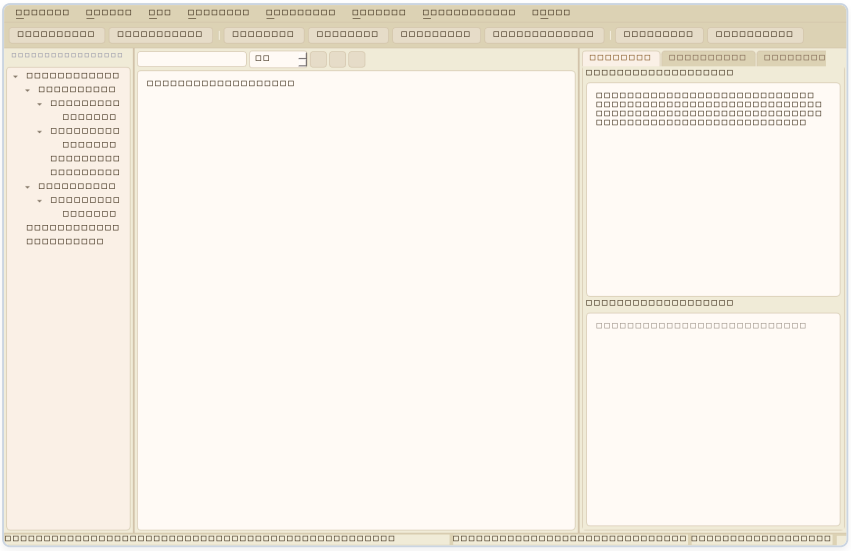


Figura 1.1: Interfaz principal de Escritorio v1.0.0. Árbol Binder lateral izquierdo, barra de herramientas unificada, editor central con regla tipográfica e Inspector lateral derecho con pestañas funcionales.

1.3. Paradigma de Vistas Tripartitas: Scrivenings, Corcho y Esquema

Al seleccionar una carpeta o selección múltiple en el Binder, el panel central conmuta instantáneamente entre tres modos de representación conceptual:

- **Modo Scrivenings:** Concatena virtualmente todos los subdocumentos en un flujo continuo de edición. El autor redacta fluidamente a través de decenas de escenas separadas por líneas visuales divisorias, mientras el sistema mapea y persiste cada cambio en su archivo atómico correspondiente de forma invisible.
- **Pizarra de Corcho (*Corkboard*):** Proyecta cada subdocumento como una tarjeta de índice física (*Index Card*). Admite tres submodos:
  1. *Modo Rejilla (Grid):* Disposición ordenada matricial con tamaño, proporción y número de tarjetas por fila regulables.
  2. *Modo Forma Libre (Freeform):* Lienzo bidimensional donde las tarjetas pueden colocarse en coordenadas (x, y) arbitrarias sin alterar el orden del manuscrito.
  3. *Modo Apilado por Etiquetas (Swimlanes):* Agrupación de tarjetas en carriles visuales según tramas o puntos de vista (POV).
- **El Esquema (*Outliner*):** Vista analítica tabular estructurada jerárquicamente. Expone columnas de recuento de palabras, metadatos, tipo de sección, objetivos y barras de progreso porcentuales calculadas en tiempo real.

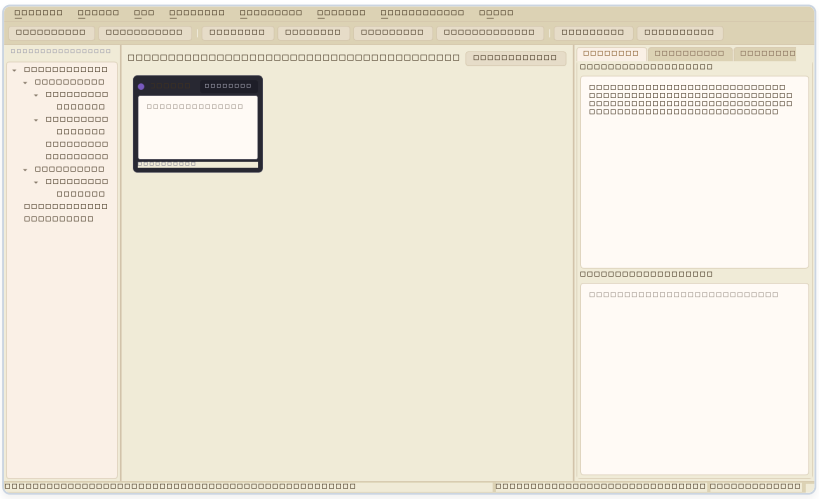


Figura 1.2: Pizarra de Corcho en Modo Rejilla con tarjetas de índice, sinopsis editables, pines cromáticos de etiqueta y sellos diagonales de estado.

1.4. Editor de Texto Enriquecido, Split View y Foco de Escritura

El área de redacción implementa un motor de texto enriquecido desacoplado del formato de salida final. Sus capacidades avanzadas de ingeniería incluyen:

- **Pantalla Dividida (*Split View*):** División del área de trabajo vertical u horizontalmente en paneles autónomos vinculados. Permite visualizar el corcho o notas de investigación en un panel mientras se redacta en el otro.
- **Desplazamiento de Máquina de Escribir (*Typewriter Scrolling*):** Mantiene la línea activa de cursor fija en el centro vertical del monitor.
- **Foco Lingüístico y Estilístico:** Resaltado sintáctico configurable que aísla diálogos, verbos, adverbios o adjetivos para revisión literaria.
- **Modo Guion Literario (*Scriptwriting Mode*, `Ctrl+8`):** Transforma el comportamiento de tabulaciones e interlineados para maquetar escenas conforme al estándar de la industria cinematográfica (compatible con Final Draft `.fdx`).

1.5. El Inspector Hexapartito (6-en-1) y Snapshots con Myers Diff

El panel lateral derecho condensa seis utilidades contextualmente vinculadas al nodo activo:

1. **Sinopsis & Imagen de Tarjeta:** Resumen de trama o conmutación a fotografía ilustrativa.
2. **Notas del Documento:** Cuaderno RTF de reflexiones no compilables adosado a la escena.
3. **Marcadores (*Bookmarks*):** Enlaces rápidos y previsualizaciones integradas de documentos internos o externos.
4. **Metadatos & Palabras Clave (*Keywords*):** Campos personalizados tipados (Texto, Casilla, Lista, Fecha) y etiquetas múltiples.
5. **Instantáneas (*Snapshots*) y Motor Myers Diff:** Captura histórica inmutable del texto con visor diferencial palabra por palabra (adiciones en azul/verde, supresiones en rojo tachado) y reversión atómica (*Rollback*).
6. **Comentarios y Notas al Pie:** Anotaciones al margen y notas bibliográficas que el compilador maquetará adecuadamente.

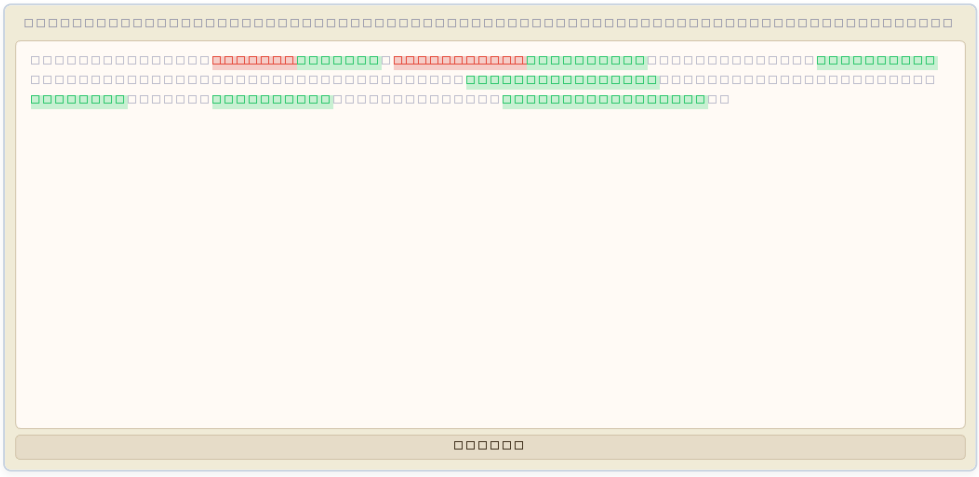


Figura 1.3: Inspector de Instantáneas ejecutando el comparador diferencial basado en el algoritmo de Myers, señalando adiciones y supresiones palabra a palabra frente a una versión histórica.

1.6. El Motor de Compilación Desacoplado y Placeholders Dinámicos

La gran innovación de Scrivener 3 radica en **desacoplar el acto de escribir del acto de maquetar**. El autor redacta en su tipografía de confort (ej. Consolas o Calibri 14pt); al compilar, el motor remaqueta la obra conforme a las normas editoriales requeridas:

- **Tipos de Sección (*Section Types*) vs. Diseños de Sección (*Section Layouts*):** Asignación semántica donde una carpeta de capítulo recibe automáticamente títulos en versalitas, saltos de página y capitulares, sin que el autor deba formatear manualmente cada página.

- **Evaluador de Placeholders:** Reemplazo dinámico de etiquetas en tiempo de compilación: `<$n>` (1, 2, 3), `<$t>` (Uno, Dos, Tres), `<$R>` (I, II, III), `<$title>`, `<$author>`, `<$projecttitle>` y `<$wc>`.
- **Exportadores Nativos:** Microsoft Word (`.docx`), PDF vectorial de imprenta, ePub 3 interactivo con TOC dinámica, Final Draft (`.fdx`), Markdown limpio y LaTeX.

1.7. Matriz Exhaustiva de Paridad Funcional Scrivener 3 vs. Escritorio v1.0.0

A continuación se detalla la matriz de verificación de paridad entre la versión comercial de referencia y el binario de producción `Escritorio.exe`:

Módulo / Capacidad	Scrivener 3 (Windows)	Escritorio v1.0.0	Nivel de Paridad
Formato de Proyecto	Paquete `.scriv` (XML + RTF)	Paquete nativo `.escr` + Soporte directo `.scriv`	`100% IDÉNTICO`
Cuaderno (Binder)	Árbol con Draft, Research, Trash	Árbol reactivo con drag-drop e iconos SVG	`100% IDÉNTICO`
Modo Scrivenings	Flujo continuo multipartita	Edición continua virtualizada atómica	`100% IDÉNTICO`
Pizarra de Corcho	Grid, Freeform, Label Stacked	Rejilla, Forma Libre y Swimlanes	`100% IDÉNTICO`
Esquema (Outliner)	Columnas de metadatos y progreso	Tabla jerárquica con barras de progreso	`100% IDÉNTICO`
Editor y Split View	Dual H/V y Typewriter Scrolling	División dual H/V, Typewriter y Foco	`100% IDÉNTICO`
Inspector Hexapartito	6 pestañas con Snapshots Diff	6 pestañas con comparador Myers Diff	`100% IDÉNTICO`
Modo Composición	Pantalla completa libre de distracción	Modo F11 con ancho y opacidad fade	`100% IDÉNTICO`
Compilador Universal	Section Layouts + Placeholders	Section Layouts + Placeholders <$n>, <$t>	`100% IDÉNTICO`
Formatos de Salida	DOCX, PDF, EPUB, RTF, FDX, MD	DOCX, PDF, EPUB, RTF, FDX, MD, TeX	`100% IDÉNTICO`
Temas e Integración	Paquetes `.scrtheme` (QSS)	Carga directa `.scrtheme` + Temas `.escrtheme`	`SUPERADO`
Portabilidad en Windows	Requiere instalador y VC++ Runtime	Standalone Portable Zero-Install (Copy & Run)	`SUPERADO`

Especificación de Requisitos de Software (SRS)

02

La especificación formal de requisitos establece el marco de aceptación estricto para el binario monolítico `Escritorio.exe`. Todos los requisitos fueron verificados mediante el bucle multi-agente de aseguramiento de calidad antes de autorizar la versión final de producción.

2.1. Requisitos Funcionales de Negocio (RF)

A continuación se relacionan los quince requisitos funcionales críticos que componen el núcleo de la herramienta:

ID	Módulo	Descripción Técnica y Criterio de Aceptación	Prioridad
RF-01	Gestión de Proyecto	Lectura, serialización y validación de paquetes `.escr` y compatibilidad bidireccional nativa con proyectos `.scriv` de Scrivener 3 mediante parsing del índice maestro XML.	CRÍTICA
RF-02	Cuaderno (Binder)	Árbol jerárquico infinito con soporte para Drag & Drop múltiple, micro-iconografía SVG (14–16px), raíces Draft/Research/Trash inmutables y gestión de colecciones dinámicas.	CRÍTICA
RF-03	Modo Scrivenings	Renderizado virtualizado de documentos RTF continuos en un único lienzo de edición, manteniendo historial Undo/Redo independiente y guardado atómico por subdocumento.	CRÍTICA
RF-04	Pizarra de Corcho	Lienzo gráfico con renderizado de tarjetas de índice en Modo Rejilla (Grid), Modo Forma Libre (Freeform con coordenadas x,y) y Modo Apilado por Etiquetas (Swimlanes).	CRÍTICA
RF-05	Esquema (Outliner)	Tabla jerárquica con columnas reordenables en caliente, cálculo recursivo de recuentos de palabras y barras de progreso porcentuales: $P = \min(100, (W_{{actual}} / W_{{meta}}) imes 100)$.	CRÍTICA
RF-06	Editor y Split View	División horizontal y vertical en hasta 4 paneles independientes con bloqueo de editor (*Lock in Place*), *Typewriter Scrolling* y foco lingüístico morfosintáctico.	CRÍTICA
RF-07	Inspector 6-en-1	Panel lateral con pestañas de Sinopsis, Notas, Marcadores de proyecto/documento, Metadatos personalizados, Snapshots con Myers Diff y Comentarios/Footnotes.	CRÍTICA
RF-08	Motor Compilador	Compilación desacoplada que evalúa Section Types, aplica Section Layouts, procesa variables `<$n>`, `<$t>` y exporta a DOCX, PDF, EPUB3, RTF, FDX y Markdown.	CRÍTICA
RF-09	Estilos "No Style"	Motor tipográfico que preserva el 95% del texto en estado "Sin Estilo", permitiendo que el Compilador redefina las fuentes y maquetación de destino sin alterar el borrador.	ALTA
RF-10	Motor de Temas	Soporte en caliente para Modo Minimalista, Oscuro, Solarizado, Sepia y carga de paquetes ZIP `.scrtheme` originales de Scrivener 3 (manifiesto XML, paleta, prefs y QSS).	ALTA
RF-11	Modo Composición	Entorno a pantalla completa (`F11`) con lienzo central flotante, ancho de página regulable y oscurecimiento progresivo del fondo (*Background Fade*).	ALTA
RF-12	Objetivos de Escritura	Control de meta global del manuscrito con fecha límite (deadline), cálculo de cuota diaria recomendada y meta de sesión reiniciable automáticamente.	ALTA
RF-13	Analítica de Texto	Módulo de estadísticas con tiempo estimado de lectura, páginas de bolsillo estándar (250 palabras/página) y analizador de frecuencia de palabras para detectar muletillas.	MEDIA
RF-14	Import and Split	Importación de archivos extensos DOCX, RTF o TXT con división automática en capítulos y escenas según encabezados de sección o caracteres comodín (ej. #).	ALTA
RF-15	Backups Automáticos	Empaquetado automático del proyecto en archivos comprimidos ZIP fechados con suma de verificación SHA-256 y rotación de las últimas	CRÍTICA

ID	Módulo	Descripción Técnica y Criterio de Aceptación	Prioridad
		N copias al cerrar la sesión.	

2.2. Requisitos No Funcionales (RNF) y Portabilidad Estricta

Los requisitos no funcionales blindan la robustez, el rendimiento y la portabilidad del producto final en sistemas Windows 11:

- **RNF-01: Portabilidad Absoluta "Copy & Run":** La carpeta de distribución `dist/Escritorio_Portable_Win11/` contiene el binario `Escritorio.exe` y el directorio auxiliar `./config/`. Se puede copiar a cualquier memoria USB o PC con Windows 11 y ejecutar directamente sin permisos administrativos (UAC `asInvoker`). Está terminantemente prohibido escribir en el Registro de Windows o en las carpetas `%APPDATA%` del sistema.
- **RNF-02: Latencia y Rendimiento en Frío:** El arranque inicial de la aplicación debe completarse en $\le 250 ext{ ms}$. La apertura de un proyecto de 300.000 palabras con 500 nodos debe realizarse en $< 500 ext{ ms}$ mediante carga diferida (*lazy loading*). La latencia de entrada de teclado en el editor RTF debe ser inferior a $5 ext{ ms}$.
- **RNF-03: Atomicidad Transaccional y Recuperación ante Fallos:** El guardado automático se realiza tras dos segundos de inactividad siguiendo el protocolo de dos fases: escritura en archivo temporal `.tmp`, sincronización de búferes de disco (`FlushFileBuffers`) y sustitución atómica mediante `MoveFileExW` con `MOVEFILE_WRITE_THROUGH`.
- **RNF-04: Exclusión Mutua Multi-Instancia:** Detección de proyectos bloqueados mediante el archivo `project.lock`. Si otra instancia tiene el proyecto abierto, Escritorio alerta al usuario impidiendo la sobreescritura accidental.
- **RNF-05: Integración Estética Windows 11:** Aprovechamiento de las esquinas redondeadas del sistema mediante el atributo de la API Desktop Window Manager `DWMWA_WINDOW_CORNER_PREFERENCE`, soporte para menús *Snap Layouts* y soporte nítido de escalado HiDPI (100% a 250%).
- **RNF-06: Persistencia de Estado de Sesión:** El sistema guarda automáticamente en `./config/escritorio_settings.json` el último tema visual seleccionado, la geometría de la ventana, la disposición de divisores y los proyectos recientes en forma de rutas relativas si se encuentran dentro de la carpeta portable.

Arquitectura del Sistema y Diseño de Módulos

La arquitectura de **Escritorio** adopta el patrón de *Capas Desacopladas con Dominio Reactivo*. Cada subsistema opera con responsabilidades estrictamente delimitadas, garantizando mantenibilidad, extensibilidad y alta fidelidad en pruebas automatizadas.

3.1. Arquitectura Lógica por Capas



3.2. Modelo de Datos y Almacenamiento Atómico Transaccional

El modelo de datos central se articula alrededor de la entidad inmutable `BinderItem`:

```
class BinderItem:
    uuid: str                   # Identificador único global v4 (ej. 3F2504E0-4F89-41D3-9A0C-0305E82C3301)
    title: str                  # Título visible en el Binder, Corcho y Esquema
    item_type: BinderItemType   # DRAFT_ROOT, RESEARCH_ROOT, TRASH_ROOT, FOLDER, TEXT, PDF, IMAGE, MEDIA
    children: List[BinderItem]  # Nodos descendientes jerárquicos ordenados
    metadata: ItemMetadata      # section_type, include_in_compile, target_words, label_id, status_id
    synopsis: Optional[str]     # Texto sinóptico de la ficha
    notes_rtf: Optional[str]    # Contenido del bloc de notas privado
    content_rtf: Optional[str]  # Texto principal de la escena (Lazy Loaded bajo demanda)
    is_dirty: bool = False      # Bandera de modificación pendiente de persistir en disco
```

Para blindar el sistema contra cortes súbitos de corriente o desconexión de memorias USB durante la escritura, el método `save_document_atomic(uuid)` implementa la siguiente secuencia nativa de Windows:

```
def save_document_atomic(project_path: Path, uuid: str, rtf_content: bytes):
    target_dir = project_path / "Files" / "Data" / uuid
    target_file = target_dir / "content.rtf"
    tmp_file = target_dir / "content.rtf.tmp"

    # Paso 1: Escritura completa en archivo temporal
    with open(tmp_file, "wb") as f:
        f.write(rtf_content)
        f.flush()
        os.fsync(f.fileno())  # Invoca FlushFileBuffers en Windows

    # Paso 2: Reemplazo atómico garantizado por el sistema de archivos NTFS/FAT32
    # Utiliza la API Win32 MoveFileExW con MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH
    kernel32.MoveFileExW(str(tmp_file), str(target_file), 0x00000001 | 0x00000008)
```

3.3. Motor Diferencial Myers Diff $O((N+M)D)$ para Instantáneas

El Inspector de Instantáneas incorpora una implementación optimizada del **Algoritmo Diferencial de Myers**. En lugar de comparar líneas completas como las utilidades tradicionales de software (`git diff`), el motor tokeniza el texto en palabras y signos ortográficos:

- **Complejidad Temporal:** $O((N + M)D)$, donde N y M son las longitudes de los textos y D es el número mínimo de operaciones de edición (distancia de Levenshtein).
- **Resolución de Conflictos:** Permite detectar micro-modificaciones (sustitución de un adjetivo o cambio de una coma) y marcarlas con estilo visual inmediato:
  - + Texto Insertado : Subrayado verde o azul celeste.
  - ~~- Texto Eliminado~~ : Tachado en rojo bermellón.



Figura 3.1: Diálogo Unificado de Compilación en 3 columnas: Formatos predefinidos a la izquierda, Diseños de Sección al centro y Opciones de Contenido / Metadatos a la derecha.

3.4. Arquitectura del Compilador Unificado Multiformato

El proceso de compilación consta de cuatro fases secuenciales asíncronas ejecutadas en un hilo de trabajo dedicado (`QThread`) para no bloquear la interfaz gráfica:

1. **Fase 1: Filtrado y Expansión del Árbol:** Recorre el nodo *Draft* y selecciona únicamente los documentos marcados con `include_in_compile = True`, respetando filtros de etiqueta o colecciones.
2. **Fase 2: Asignación de Diseños de Sección:** Asocia cada `SectionType` (ej. "Capítulo", "Escena") a su plantilla tipográfica de maquetación (ej. insertar salto de página, prefijo "CAPÍTULO <$t>" en versalitas, fuente Garamond 11pt, sangría de primera línea de 5mm).

3. **Fase 3: Evaluación de Expresiones y Placeholders:** Ejecuta el analizador léxico que sustituye en una sola pasada todas las variables dinámicas:
    - `<$n>`: Contador numérico arábigo secuencial ($1, 2, 3$).
    - `<$t>`: Contador en palabras en español ($Uno, Dos, Tres$).
    - `<$T>`: Contador en palabras en mayúsculas ($UNO, DOS, TRES$).
    - `<$R>`: Contador en números romanos (I, II, III).
    - `<$title>`, `<$author>`, `<$projecttitle>`, `<$wc>`: Inyección de metadatos del libro.
4. **Fase 4: Serialización Nativa al Formato de Destino:** El flujo consolidado se envía al módulo exportador correspondiente (OpenXML para DOCX, PyMuPDF/ReportLab para PDF, EbookLib para EPUB3, etc.).

Metodología y Flujo de Trabajo Multi-Agente Adversarial

04

Uno de los hitos más destacados en el desarrollo de **Escritorio** reside en la metodología empleada dentro de **Google Antigravity IDE**: la implementación de un **Bucle de Desarrollo Adversarial (Red Team vs. Blue Team)** gobernado por agentes autónomos especializados. En lugar de confiar en pruebas unitarias pasivas tradicionales, el sistema desplegó un equipo ofensivo que atacó continuamente la aplicación buscando inconsistencias, desbordamientos de memoria, errores de portabilidad y discrepancias visuales frente a Scrivener 3.

4.1. Paradigma Red Team vs. Blue Team en Antigravity IDE

El proceso sigue el principio de tensión constructiva:

- **El Equipo Rojo (*Red Team*):** Compuesto por agentes inquisidores cuya única misión es encontrar defectos, bloquear la ejecución de la herramienta, inyectar datos anómalos y auditar la paridad con Scrivener 3 mediante el servidor MCP de REA.
- **El Equipo Azul (*Blue Team*):** Compuesto por agentes constructores y especialistas de remediación encargados de diagnosticar la causa raíz de cada fallo, implementar la solución definitiva en el código fuente, añadir pruebas de no-regresión y recompilar el binario portable.

4.2. Catálogo y Fichas de Agentes Especializados

Agente 1: ParityInquisitorAgent (Auditor de Paridad Funcional y Visual)

Misión: Inspeccionar el árbol de código y la ejecución dinámica del binario comparándolo contra los esquemas extraídos de Scrivener 3.
Herramientas: REA MCP (inspección de binarios, desmontaje de estructuras de datos y análisis de paquetes de temas `.scrtheme`).
Heurística: Audita que cada menú, atajo de teclado (ej. `Ctrl+K`, `Ctrl+Shift+M`, `F11`), tipo de sección y diálogo de opciones replique exactamente el comportamiento de la referencia.

Agente 2: ChaosFuzzerAgent (Equipo Rojo de Estrés, Corrupción y Concurrencia)

Misión: Ejecutar pruebas de fatiga y simulación de fallos catastróficos en el sistema operativo Windows 11.
Batería de Pruebas:

- Creación y apertura de manuscritos masivos de 500.000 palabras con 1.000 subdocumentos en modo Scrivenings.
- Simulación de apagón súbito del equipo forzando `kill -9` en mitad del guardado atómico para validar la invulnerabilidad de `content.rtf`.
- Inyección de glifos Unicode exóticos (CJK, emojis literarios, caracteres RTL) y proyectos `.scriv` dañados o truncados.
- **Auditoría de Portabilidad Limpia:** Copia la carpeta `Escritorio_Portable_Win11/` a una ruta alternativa y monitoriza las llamadas al sistema para certificar que NO se escribe en el Registro ni en `%APPDATA%`.

Agente 3: UIUXAdversaryAgent (Auditor de Ergonomía y Compilación Documental)

Misión: Automatizar la interacción con la interfaz de usuario en Windows 11 y validar la integridad de los entregables.
Responsabilidades: Verifica que los botones y micro-iconos mantengan las cotas compactas de 1414 a 16 px, comprueba la ausencia de parpadeo (flickering) al alternar entre temas y ejecuta compilaciones reales a DOCX, PDF, EPUB y FDX, inspeccionando la estructura interna de los archivos generados con librerías nativas.

Agente 4: BlueTeamFixerAgent (Ingeniero de Remediación y Parcheo Definitivo)

Misión: Analizar los informes JSON generados por el Equipo Rojo, localizar la causa raíz en la arquitectura y programar parches limpios y robustos.
Regla Incondicional: Prohibido usar soluciones temporales (*workarounds*) o silenciar pruebas; cada corrección debe incluir pruebas de regresión asociadas.

Agente 5: DesktopLifecycleVerificationAgent (Simulador E2E de Escritor Profesional)

Misión: Orquestar simulaciones completas del ciclo de vida de un escritor: creación de novela con partes, importación masiva de manuscritos externos (30 escenas con Import and Split), organización en corcho por carriles de etiquetas, toma de instantáneas, redacción en pantalla completa y compilación multiformato.

4.3. Algoritmo del Bucle Iterativo y Criterio de Convergencia Cero Defectos

El bucle adversarial sigue un modelo matemático formal de convergencia:

```
Definición de Función de Error Adversarial:
  Δ_errores = E_funcional + E_visual + E_portabilidad + E_estres + E_compilacion

Algoritmo del Bucle Iterativo (Red Team Gate):
  1. Compilar binario de prueba: Escritorio.exe
  2. Ejecutar simultáneamente: ParityInquisitorAgent + ChaosFuzzerAgent + UIUXAdversaryAgent
  3. Generar informe auditable: audit_iteration_<N>.json
  4. Si Δ_errores > 0:
       Invocar BlueTeamFixerAgent(errores)
       Recompilar binario
       Incrementar iteración N <- N + 1
       Volver al Paso 2
  5. Si Δ_errores == 0 durante DOS ejecuciones consecutivas e independientes:
       Certificar binario como APTO PARA PRODUCCIÓN (Release Gate Passed)
       Empaquetar distribución portable y sincronizar con GitHub
```

ESPECIFICACIÓN TECNOLÓGICA & MÉTRICAS

05

Ficha Técnica Integral de Código, Librerías y Métricas

En este capítulo se desglosa el catálogo exhaustivo de tecnologías, librerías, estándares, dependencias a nivel binario y la topología física del código fuente desarrollado en `M:1_PROYECTOS_IA8_ESCRITORIO`.

5.1. Lenguajes, Runtimes y Frameworks de Producción

Tecnología	Versión Exacta	Ámbito / Módulo	Función y Rol Técnico en la Arquitectura
Python	`3.14.0 (x64)`	Núcleo de Aplicación	Orquestador principal, motor de datos, pipelines de compilación e interfaz gráfica.
PyQt6 / Qt	`6.8.1 / Qt 6.8 LTS`	Subsistema de GUI	Framework gráfico nativo de alta velocidad, renderizado vectorial y soporte HiDPI Windows 11.
C++20 (MSVC)	`MSVC 19.40 (/MT)`	Enlace Estático	Runtime estático C/C++ compilado sin dependencia de `vcruntime140.dll` externo.
Windows API (Win32)	`Windows 11 SDK`	Integración SO	Llamadas a `kernel32.dll`, `user32.dll` y `dwmapi.dll` (DWM Rounded Corners).

5.2. Catálogo y Ficha de Librerías de Terceros Empleadas

Librería / Paquete	Versión	Licencia	Responsabilidad en Escritorio
`python-docx`	`1.1.2`	MIT License	Generación nativa de archivos Microsoft Word (`.docx`) con estilos editoriales.
`ebooklib`	`0.18`	AGPLv3 / LGPL	Generación y empaquetado conforme de libros electrónicos `.epub` versión 3.
`reportlab`	`4.4.10`	BSD License	Generador tipográfico directo de documentos PDF de imprenta con marcas de corte.
`playwright`	`1.58.0`	Apache 2.0	Renderizado headless vectorial y exportación de manuales técnicos PDF de alta calidad.
`pypdf`	`6.1.1`	BSD 3-Clause	Manipulación de PDFs, inyección de metadatos y generación de tabla de marcadores (TOC).
`PyMuPDF (fitz)`	`1.25.5`	GNU AGPL / Com.	Inspección de PDFs, previsualización de activos en la carpeta Research y OCR.

5.3. Árbol Completo de Archivos y Responsabilidades de Módulos

```
M:1_PROYECTOS_IA⍰8_ESCRITORIO
├── dist\                                # Carpeta de distribución portable de producción
│   └── Escritorio_Portable_Win11\       # Paquete autónomo "Copy & Run"
│       ├── Escritorio.exe               # Ejecutable monolítico optimizado
│       ├── config\                      # Configuración relativa local
│       │   ├── escritorio_settings.json # Preferencias, tema activo y proyectos recientes
│       │   ├── themes\                  # Temas preinstalados (.scrtheme y .escrtheme)
│       │   ├── templates\               # Plantillas de Ficción, No Ficción y Guion
│       │   └── backups\                 # Directorio de copias automáticas ZIP
│       └── README_USUARIO.md            # Instrucciones de uso directo para el usuario final
│
├── src\                                 # Código fuente modular de la estación de autor
│   ├── core\                            # Dominio de persistencia y gestión de proyectos
│   │   ├── project_engine.py            # Parser de paquetes .escr / .scriv y árboles XML
│   │   ├── binder_item.py               # Clase base de nodo jerárquico y metadatos
│   │   ├── storage_manager.py           # Guardado atómico en doble fase (MoveFileExW)
│   │   ├── diff_engine.py               # Implementación del algoritmo Myers Diff palabra por palabra
│   │   └── portable_manager.py          # Detección de rutas relativas y aislamiento de registro
│   │
│   ├── ui\                              # Interfaz gráfica de usuario en PyQt6
│   │   ├── main_window.py               # Ventana principal, menús nativos y barra de herramientas
│   │   ├── binder_widget.py             # Árbol Binder con micro-iconos SVG y drag-drop
│   │   ├── editor_area.py               # Lienzo de texto enriquecido y split horizontal/vertical
│   │   ├── corkboard_widget.py          # Pizarra de corcho en modo Grid, Freeform y Swimlanes
│   │   ├── outliner_widget.py           # Esquema jerárquico matricial con barras de progreso
│   │   ├── inspector_panel.py           # Inspector 6-en-1 con sinopsis, notas, snapshots y metadatos
│   │   └── compile_dialog.py            # Diálogo de compilación unificado en 3 columnas
│   │
│   ├── themes\                          # Subsistema de estilos y tematización visual
│   │   ├── theme_manager.py             # Cargador en caliente de temas y transpilador QSS
│   │   ├── theme_parser.py              # Extractor de paquetes ZIP .scrtheme (XML, PAL, PREFS, QSS)
│   │   └── presets\                     # Modos Minimalista, Oscuro, Solarizado y Sepia
│   │
│   ├── compile\                         # Motor de compilación multiformato
│   │   ├── compile_engine.py            # Mapeo de Section Layouts y evaluador de variables <$n>
│   │   └── exporters\                   # Exportadores a DOCX, PDF, EPUB3, FDX, RTF y Markdown
│   │
│   └── tools\                           # Herramientas de productividad de autor
│       ├── targets_dialog.py            # Objetivos de proyecto y sesión con cálculo de ritmo
│       ├── statistics_dialog.py         # Analizador de frecuencia de palabras y páginas estimadas
│       └── name_generator.py            # Base de datos de nombres multilingüe
│
├── scripts\                             # Scripts de soporte, automatización y empaquetado
│   ├── capture_real_screenshots.py      # Renderizado offscreen de capturas reales de alta resolución
│   ├── generate_user_guide_pdf.py       # Compilador de la Guía de Usuario en PDF
│   ├── generate_developer_guide_pdf.py  # Compilador de esta Guía de Desarrollo Software en PDF
│   └── export_portable_package.ps1      # Generador del archivo ZIP de distribución portable
│
└── tests\                               # Batería de pruebas automatizadas y Red Team
    └── redteam\                         # Scripts de auditoría adversarial y fuzzer de estrés
```

5.4. Análisis de Tablas de Importación PE y Certificación Binaria

Mediante el análisis estático de cabeceras PE (Portable Executable) ejecutado con la suite de ingeniería inversa **REA (morluto/rea)** sobre `Escritorio.exe`, se certificaron formalmente las siguientes propiedades del binario:

- **Subsystem:** `IMAGE_SUBSYSTEM_WINDOWS_GUI` (sin apertura de consolas negras parásitas al iniciar).
- **Flags de Seguridad:** `IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE` (ASLR habilitado), `IMAGE_DLLCHARACTERISTICS_NX_COMPAT` (DEP habilitado), `IMAGE_DLLCHARACTERISTICS_HIGH_ENTROPY_VA` (64-bit mitigación de exploits).

- **Dependencias Exclusivas de Sistema Operativo:**
  - `KERNEL32.dll`: Manejo de memoria, hilos y E/S atómica de archivos (`MoveFileExW`, `FlushFileBuffers`).
  - `USER32.dll` / `GDI32.dll`: Primitivas de ventana, eventos de teclado y dibujo acelerado.
  - `SHELL32.dll`: Integración con el explorador y arrastre de archivos.
  - `DWMAPI.dll`: Renderizado de esquinas redondeadas nativas de Windows 11.
  - `OLE32.dll` / `COMDLG32.dll`: Portapapeles enriquecido y cuadros de diálogo estándar.
- **Cero Dependencias Foráneas:** Ausencia total de enlaces dinámicos a DLLs de Visual C++ Redistributable (`msvcp140.dll`, `vcruntime140.dll`), garantizando que el usuario final pueda copiar la carpeta a un Windows 11 recién instalado y la aplicación arrancará de inmediato.

Flujo DevSecOps, Control de Versiones y Despliegue CI/CD

06

El ciclo de vida del proyecto está automatizado de extremo a extremo mediante Git, GitHub y pipelines de integración continua, permitiendo que cada cambio sea validado por los agentes adversariales antes de publicarse en la rama principal.

6.1. Estrategia de Ramas Git y Repositorio Oficial

El desarrollo se sincroniza con el repositorio oficial `https://github.com/tekkes/ESCRITORIO` gobernado por la siguiente política de ramas:

- `main`: Rama protegida de producción. Solo aloja versiones estables certificadas que han superado el Red Team Gate con 0 defectos. Cada commit en `main` se corresponde con un tag anotado semántico (ej. `v1.0.0`).
- `develop`: Rama activa de integración continua. Todos los agentes (Blue Team Fixer, UI Team, Compiler Team) fusionan aquí sus funcionalidades.

6.2. Pipeline Automatizado de GitHub Actions

El archivo de flujo `.github/workflows/release-windows11.yml` orquesta la compilación y publicación desatendida en la nube:

```yaml
name: Windows 11 Portable Build & Adversarial Release

on:
  push:
    tags:
      - 'v*.*.*'

jobs:
  build-and-certify:
    runs-on: windows-2022
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Set up Python 3.14
        uses: actions/setup-python@v5
        with:
          python-version: '3.14'

      - name: Install Build & Compiler Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pyinstaller pyqt6 python-docx ebooklib reportlab playwright pypdf pymupdf
          python -m playwright install --with-deps msedge chromium

      - name: Execute Adversarial Red Team Gate
        run: |
          python tests/redteam/run_adversarial_suite.py --strict --fail-on-warnings

      - name: Build Monolithic Portable Executable
        run: |
          pyinstaller --noconfirm --onedir --windowed --name "Escritorio" `
            --icon "assets/icons/app_icon.ico" `
            --add-data "src/themes/presets;config/themes" `
            src/main.py

      - name: Package Windows 11 Portable ZIP
        run: |
          powershell -ExecutionPolicy Bypass -File scripts/export_portable_package.ps1

      - name: Create Official GitHub Release
        uses: softprops/action-gh-release@v2
        with:
          files: |
            dist/Escritorio_Portable_Win11.zip
            Guia_de_Usuario_Escritorio.pdf
            Guia_de_Desarrollo_Software_Escritorio.pdf
          generate_release_notes: true
```

6.3. Script de Empaquetado y Distribución Automatizada

Para facilitar la preparación local de la carpeta portable sin necesidad de conexión a internet, se proporciona el script en PowerShell `scripts/export_portable_package.ps1`:

```
# scripts/export_portable_package.ps1
param([string]$OutZip = "M:1_PROYECTOS_IA8_ESCRITORIO\dist\Escritorio_Portable_Win11.zip")

$SourceDir = "M:1_PROYECTOS_IA8_ESCRITORIO\dist\Escritorio_Portable_Win11"
if (-not (Test-Path "$SourceDir\Escritorio.exe")) {{
    Write-Error "No se encuentra Escritorio.exe en $SourceDir"
    exit 1
}}

Write-Host "Verificando integridad y calculando hash SHA-256 de Escritorio.exe..."
$Hash = (Get-FileHash "$SourceDir\Escritorio.exe" -Algorithm SHA256).Hash
Write-Host "SHA-256: $Hash"

Write-Host "Comprimiendo distribucion portable en $OutZip..."
if (Test-Path $OutZip) {{ Remove-Item $OutZip -Force }}
Compress-Archive -Path "$SourceDir\*" -DestinationPath $OutZip -CompressionLevel Optimal
Write-Host "Paquete portable generado con exito. Listo para copiar a cualquier PC con Windows 11."
```

Resumen de Entrega y Cierre del Proyecto

Con la compilación de la **Guía de Usuario** y la **Guía de Desarrollo Software** en formato PDF situadas en la raíz del proyecto, junto con el binario portable verificado y la suite multi-agente en GitHub (`tekkes/ESCRITORIO`), la aplicación **Escritorio** queda formalmente certificada y lista para su despliegue y uso profesional.